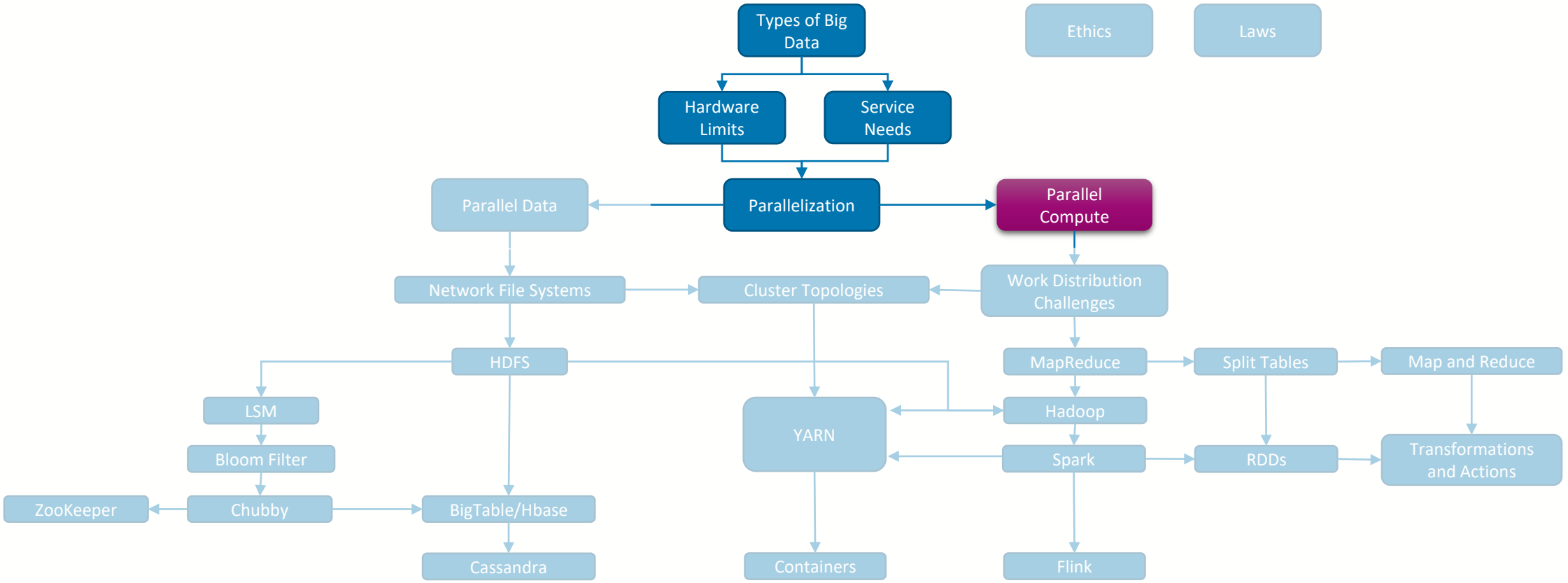

# **W1-05:** Compute Parallelization

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

University of Glasgow

# Where are we?



# Definition

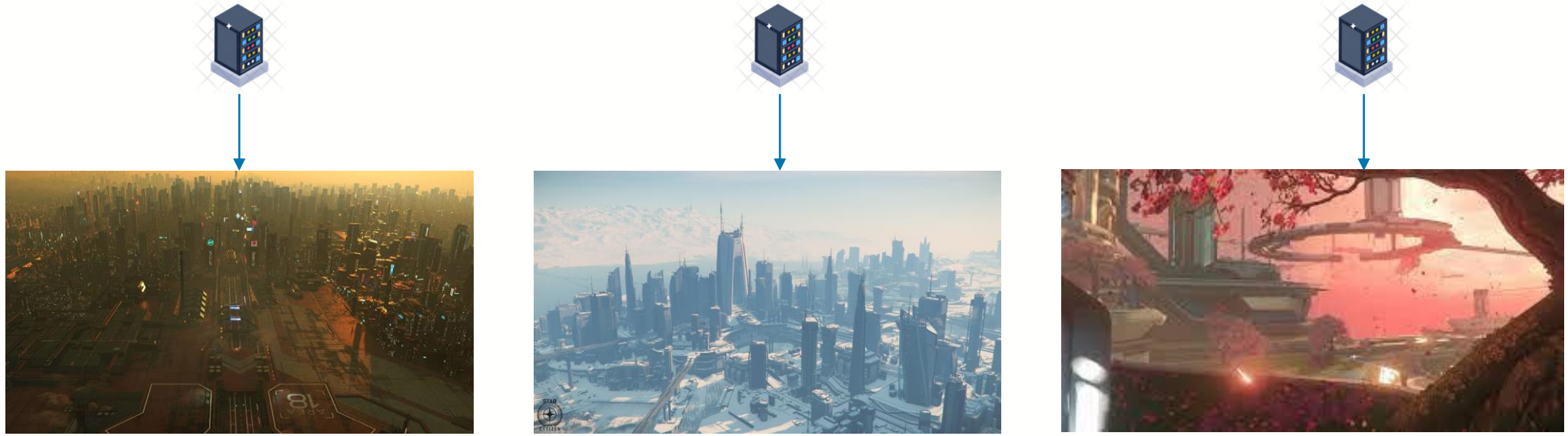
- **Parallel Computing**: Solving a (Big Data) problem by breaking down that problem into multiple tasks that can be solved simultaneously
  - These tasks can then be solved using horizontally scalable resources

- Caveat: This is not a standardized definition, resources outside of this presentation may use the terms in a different way

University of Glasgow

# Example: Star Citizen, Server Meshing

- Problem: The video game universe of Star Citizen is too large to provide server-side simulation for on one server
- Parallel Solution: Break the game world into regions and assign one server to manage each region (then move players between servers as they traverse the world)



University of Glasgow

# Where can Compute Parallelism happen?

- Within a CPU core: Modern CPU architectures can process multiple instructions simultaneously due to pipelining and superscalar support (invisible to the programmer)
  - Pipelining: Split the processor datapath into multiple successive stages (e.g. fetching, decoding, execution, etc) with instructions moving through each stage sequentially
  - Superscalar: Processors that house multiple copies of certain commonly used components (e.g. arithmetic logic units)

- Across CPU cores: Each CPU core can process different instructions, often exposed to programmers as ‘threads’

- Across CPU cores in different machines: Tasks are typically compiled and then sent to a machine with some data to execute that task on (horizontal scaling)
  - This is what we will focus on here

# Flynn's Taxonomies

- A Hardware Classification for CPUs
  - Single Instruction (SI)
    - System in which all processors execute the same instruction
  - Multiple Instruction (MI)
    - System in which different processors may execute different instructions
  - Single Data (SD)
    - System in which all processors operate on the same data
  - Multiple Data (MD)
    - System in which different processors may operate on different data

- **SISD** – Classic von Neumann architecture; serial computer
- **MIMD** – Collections of autonomous processors that can execute multiple independent programs; each of which can have its own data stream
  - Modern CPUs
- **SIMD** – Data is divided among the processors and each data item is subjected to the same sequence of instructions;
  - E.g. GPUs, Advanced Vector Extensions (AVX)
- **MISD** – Very rare
  - e.g. systolic arrays

M. J. Flynn. Some computer organizations and their effectiveness. IEEE Transactions on Computers, C-21(9):948–960, 1972.

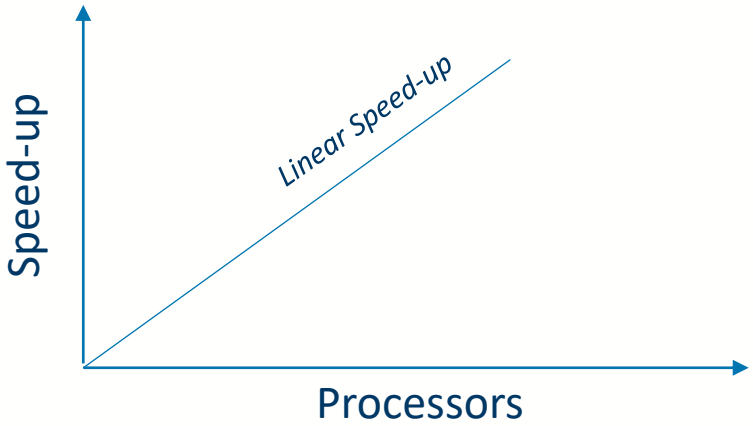
University of Glasgow

# Measuring Parallelism Effectiveness

- We typically measure the effectiveness of compute parallelism using a measure called ‘speed-up’
  - Speed-up(n,p) = Serial Time(n) / Parallel Time(n,p)
    - n = input size
    - p = number of parallel processors

- We expect that:
  - 0 < Speed-up <= p

- Linear Speed-up: Speed-up = p



Course Coordinator
**Dr. Richard McCreadie**

**Email:** richard.mccreadie@glasgow.ac.uk
**Room:** SAWB 304

**#UofGWorldChangers**
**@UofGlasgow**